

Отчёт по лабораторной работе №7

**Команды безусловного и условного переходов в Nasm.
Программирование ветвлений.**

Мещерякова София Андреевна

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
2.1	Реализация переходов в NASM	5
2.2	Изучение структуры файлы листинга	15
2.3	Выполнение заданий для самостоятельной работы	18
3	Выводы	26

Список иллюстраций

2.1	Создаем каталог и файл для лабораторной работы №7	5
2.2	Текст программы	6
2.3	Запуск программы	6
2.4	Текст программы	9
2.5	Запуск программы	10
2.6	Текст программы	10
2.7	Запуск программы	11
2.8	Создание файла lab7-2.asm	12
2.9	Текст программы	13
2.10	Запуск программы	15
2.11	Создание файла листинга	15
2.12	Открываем файл листинга	16
2.13	Файл листинга	16
2.14	Строки листинга для объяснения	17
2.15	Удаляем один из операндов инструкции mov	17
2.16	При создании файла листинга выводится сообщение об ошибке . . .	17
2.17	Файл листинга	17
2.18	Вариант	18
2.19	Текст программы	19
2.20	Запуск программы	19
2.21	Текст программы	24
2.22	Запуск программы	24
2.23	Запуск программы при (х,а): (2, 3)	25
2.24	Запуск программы при (х,а): (4, 2)	25

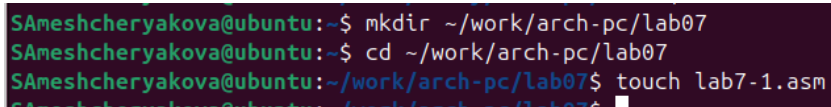
1 Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2 Выполнение лабораторной работы

2.1 Реализация переходов в NASM

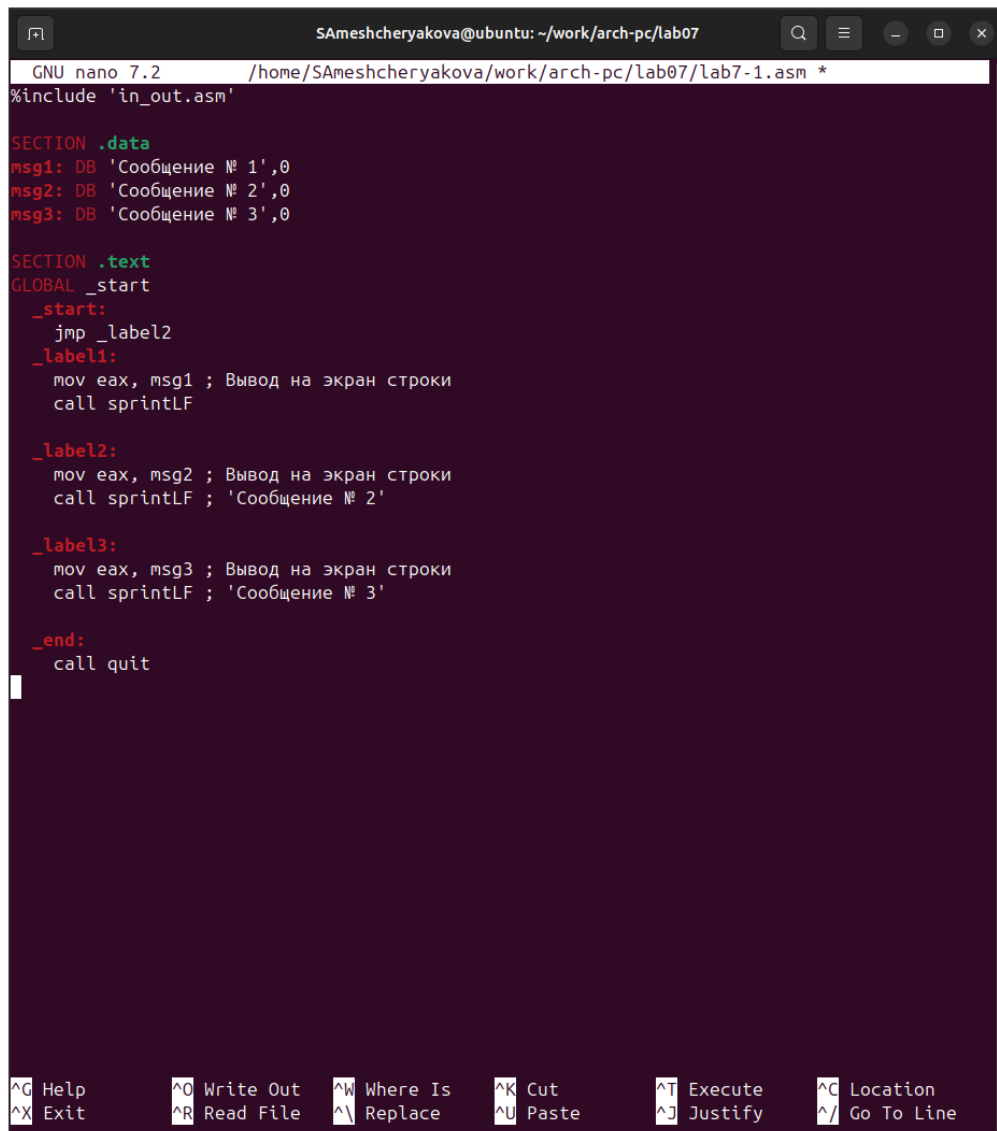
1. Создадим каталог для программ лабораторной работы № 7, перейдём в него и создадим файл lab7-1.asm (рис. 2.1).



```
SAmeshcheryakova@ubuntu:~$ mkdir ~/work/arch-pc/lab07
SAmeshcheryakova@ubuntu:~$ cd ~/work/arch-pc/lab07
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ touch lab7-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$
```

Рисунок 2.1: Создаем каталог и файл для лабораторной работы №7

2. Рассмотрим пример программы с использованием инструкции jmp. Введем в файл lab7-1.asm текст программы из листинга 7.1 (рис. 2.2).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab07/lab7-1.asm *
%include 'in_out.asm'

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:
    jmp _label2
_label1:
    mov eax, msg1 ; Вывод на экран строки
    call sprintLF

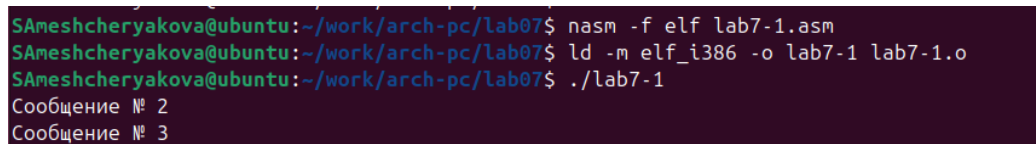
_label2:
    mov eax, msg2 ; Вывод на экран строки
    call sprintLF ; 'Сообщение № 2'

_label3:
    mov eax, msg3 ; Вывод на экран строки
    call sprintLF ; 'Сообщение № 3'

_end:
    call quit
```

Рисунок 2.2: Текст программы

Создадим исполняемый файл и запустим его (рис. 2.3).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
```

Рисунок 2.3: Запуск программы

Листинг 7.1

```

#include 'in_out.asm'

SECTION .data
msg1: DB 'XXXXXXXX № 1',0
msg2: DB 'XXXXXXXX № 2',0
msg3: DB 'XXXXXXXX № 3',0

SECTION .text
GLOBAL _start
_start:
    jmp _label2

_label1:
    mov eax, msg1
    call printf

_label2:
    mov eax, msg2
    call printf

_label3:
    mov eax, msg3
    call printf

_end:
    call quit

```

Изменим программу таким образом, чтобы она выводила сначала „Сообщение № 2“, потом „Сообщение № 1“ и завершала работу. Измените текст программы в

соответствии с листингом 7.2 (рис. 2.4).

Листинг 7.2

```
%include 'in_out.asm'

SECTION .data
msg1: DB 'XXXXXXXX № 1',0
msg2: DB 'XXXXXXXX № 2',0
msg3: DB 'XXXXXXXX № 3',0

SECTION .text
GLOBAL _start

_start:
    jmp _label2

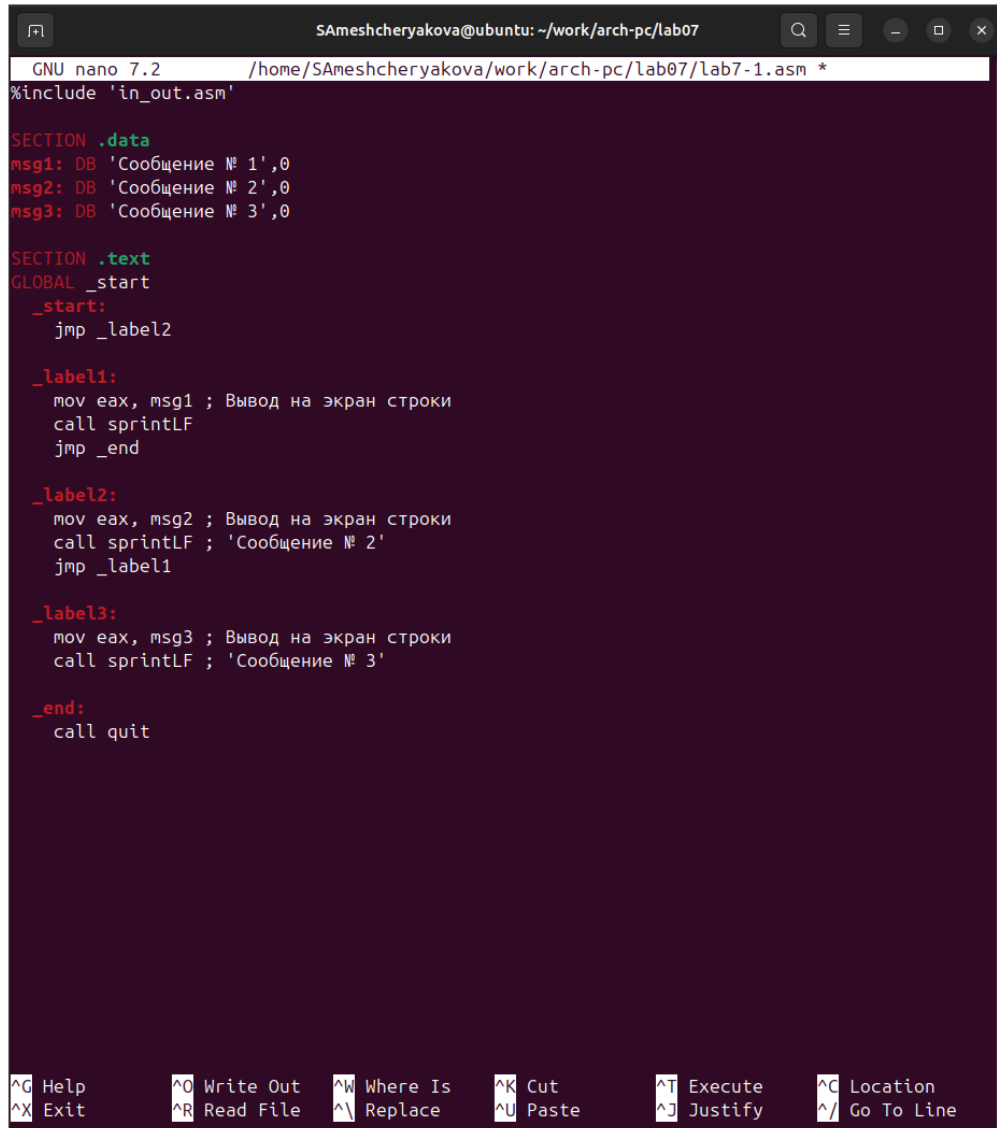
_label1:
    mov eax, msg1 ; XXXXX XX XXXXX XXXXXX
    call sprintLF
    jmp _end

_label2:
    mov eax, msg2 ; XXXXX XX XXXXX XXXXXX
    call sprintLF ; 'XXXXXXXX № 2'
    jmp _label1

_label3:
    mov eax, msg3 ; XXXXX XX XXXXX XXXXXX
    call sprintLF ; 'XXXXXXXX № 3'
```



```
_end:  
  
call quit
```



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab07/lab7-1.asm *  
%include 'in_out.asm'  
  
SECTION .data  
msg1: DB 'Сообщение № 1',0  
msg2: DB 'Сообщение № 2',0  
msg3: DB 'Сообщение № 3',0  
  
SECTION .text  
GLOBAL _start  
_start:  
    jmp _label2  
  
_label1:  
    mov eax, msg1 ; Вывод на экран строки  
    call sprintfLF  
    jmp _end  
  
_label2:  
    mov eax, msg2 ; Вывод на экран строки  
    call sprintfLF ; 'Сообщение № 2'  
    jmp _label1  
  
_label3:  
    mov eax, msg3 ; Вывод на экран строки  
    call sprintfLF ; 'Сообщение № 3'  
  
_end:  
    call quit
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

Рисунок 2.4: Текст программы

Создадим исполняемый файл и проверим его работу (рис. 2.5).

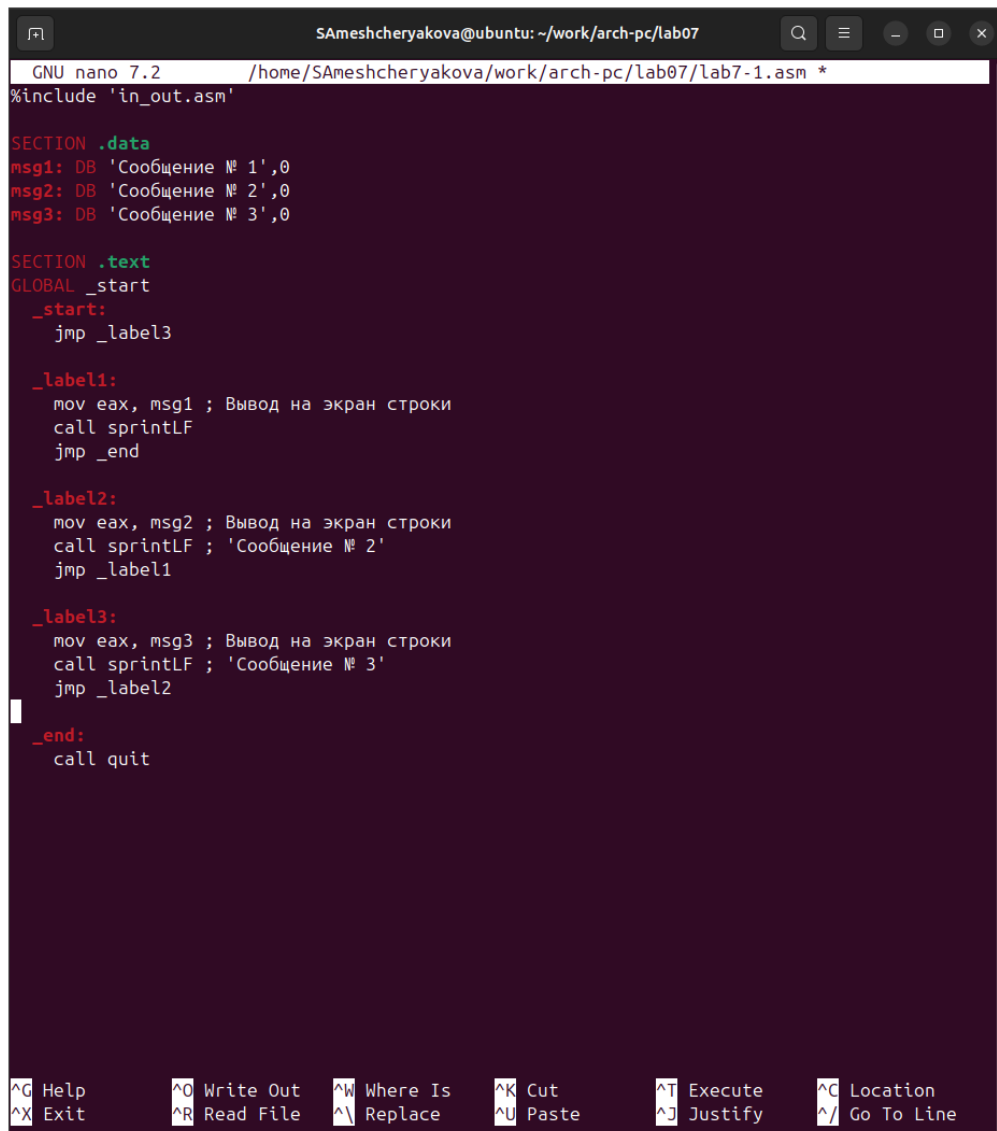
```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1

```

Рисунок 2.5: Запуск программы

Изменим текст программы, чтобы она выводила все 3 сообщения (рис. 2.6).



```

GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab07/lab7-1.asm *
%include 'in_out.asm'

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:
    jmp _label3

_label1:
    mov eax, msg1 ; Вывод на экран строки
    call sprintfLF
    jmp _end

_label2:
    mov eax, msg2 ; Вывод на экран строки
    call sprintfLF ; 'Сообщение № 2'
    jmp _label1

_label3:
    mov eax, msg3 ; Вывод на экран строки
    call sprintfLF ; 'Сообщение № 3'
    jmp _label2

_end:
    call quit

```

Рисунок 2.6: Текст программы

Создадим исполняемый файл и проверим его работу (рис. 2.7).

```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1

```

Рисунок 2.7: Запуск программы

Листинг 7.2 (изменённый)

```

#include 'in_out.asm'

SECTION .data
msg1: DB 'XXXXXXXX № 1', 0
msg2: DB 'XXXXXXXX № 2', 0
msg3: DB 'XXXXXXXX № 3', 0

SECTION .text
GLOBAL _start

_start:
    jmp _label3

_label1:
    mov eax, msg1
    call sprintLF
    jmp _end

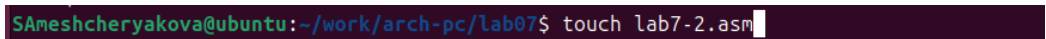
_label2:
    mov eax, msg2
    call sprintLF
    jmp _label1

```

```
_label3:
    mov eax, msg3
    call sprintf
    jmp _label2

_end:
    call quit
```

3. Создадим файл lab7-2.asm в каталоге ~/work/arch-pc/lab07 (рис. 2.8). Изучим текст программы из листинга 7.3 и введем в lab7-2.asm (рис. 2.9).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ touch lab7-2.asm
```

Рисунок 2.8: Создание файла lab7-2.asm

```

mc [SAmeshcheryakova@ubuntu]:~/work/arch-pc/lab07
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab07/lab7-2.asm *
#include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ecx,[B] ; иначе 'ecx = B'

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```

Рисунок 2.9: Текст программы

Листинг 7.3

```

#include 'in_out.asm'

section .data
    msg1 db 'XXXXXXXX B: ',0h
    msg2 db "XXXXXXXXXX XXXX: ",0h
    A dd '20'

```

```

C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
    mov eax,msg1
    call sprint
    mov ecx,B
    mov edx,10
    call sread
    mov eax,B
    call atoi
    mov [B],eax
    mov ecx,[A]
    mov [max],ecx
    cmp ecx,[C]
    jg check_B
    mov ecx,[C]
    mov [max],ecx
check_B:
    mov eax,max
    call atoi
    mov [max],eax
    mov ecx,[max]
    cmp ecx,[B]
    jg fin

```

```

    mov ecx, [B]
    mov [max], ecx
fin:
    mov eax, msg2
    call sprint
    mov eax, [max]
    call iprintLF
    call quit

```

Создадим исполняемый файл и проверим его работу при разных значениях В (рис. 2.10).

```

nasm: fatal: unable to open input file: lab7-1.asm: no such file or directory
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 15
Наибольшее число: 50
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 68
Наибольшее число: 68
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 20
Наибольшее число: 50

```

Рисунок 2.10: Запуск программы

2.2 Изучение структуры файлы листинга

4. Создадим файл листинга для программы из файла lab7-2.asm (рис. 2.11).

```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm

```

Рисунок 2.11: Создание файла листинга

Откроем файл листинга lab7-2.lst с помощью любого текстового редактора, например mcedit (рис. 2.12).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ mcedit lab7-2.lst
```

Рисунок 2.12: Открываем файл листинга

```
/home/SAmeshc~7/lab7-2.lst [----] 0 L:[185+16 201/227] *(12371/14540b) 0032 0x020 [*][X]
10 0000000A <res Ah> B resb 10
11 .....
12 section .text
13 global _start
14 _start:
15 ; ----- Вывод сообщения 'Введите B: '
16 000000E8 B8[00000000] mov eax,msg1
17 000000ED E81DFFFFFF call sprint
18 ; ----- Ввод 'B'
19 000000F2 B9[0A000000] mov ecx,B
20 000000F7 BA0A000000 mov edx,10
21 000000FC E842FFFFFF call sread
22 ; ----- Преобразование 'B' из символа в число
23 00000101 B8[0A000000] mov eax,B
24 00000106 E891FFFFFF call atoi ; Вызов подпрограммы перевода символа в ч
25 0000010B A3[0A000000] mov [B],eax ; запись преобразованного числа в 'B'
26 ; ----- Записываем 'A' в переменную 'max'
27 00000110 8B0D[35000000] mov ecx,[A] ; 'ecx = A'
28 00000116 890D[00000000] mov [max],ecx ; 'max = A'
29 ; ----- Сравниваем 'A' и 'C' (как символы)
30 0000011C 3B0D[39000000] cmp ecx,[C] ; Сравниваем 'A' и 'C'
31 00000122 7F0C jg check_B ; если 'A>C', то переход на метку 'check
32 00000124 8B0D[39000000] mov ecx,[C] ; иначе 'ecx = C'
33 0000012A 890D[00000000] mov [max],ecx ; 'max = C'
34 ; ----- Преобразование 'max(A,C)' из символа в
35 check_B:
36 00000130 B8[00000000] mov eax,max
37 00000135 E862FFFFFF call atoi ; Вызов подпрограммы перевода символа в ч
38 0000013A A3[00000000] mov [max],eax ; запись преобразованного числа в 'ма
39 ; ----- Сравниваем 'max(A,C)' и 'B' (как числа
40 0000013F 8B0D[00000000] mov ecx,[max]
41 00000145 3B0D[0A000000] cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
42 0000014B 7F0C jg fin ; если 'max(A,C)>B', то переход на 'fin',
43 0000014D 8B0D[0A000000] mov ecx,[B] ; иначе 'ecx = B'
44 00000153 890D[00000000] mov [max],ecx
45 ; ----- Вывод результата
46 fin:
47 00000159 B8[13000000] mov eax,msg2
48 0000015E E8ACFFFFFF call sprint ; Вывод сообщения 'Наибольшее число: '
49 00000163 A1[00000000] mov eax,[max]
50 00000168 E819FFFFFF call iprintLF ; Вывод 'max(A,B,C)'
51 0000016D E869FFFFFF call quit ; Выход

1Help 2Save 3Mark 4Replac 5Copy 6Move 7Search 8Delete 9PullDn 10Quit
```

Рисунок 2.13: Файл листинга

Внимательно ознакомиться с его форматом и содержимым. Подробно объяснить содержание трёх строк файла листинга по выбору (рис. 2.14).


```

23 00000101 B8[0A000000]      mov eax,B
24 00000106 E891FFFFFF      call atoi ; Вызов подпрограммы перевода символа в число
25 0000010B A3[0A000000]      mov [B],eax ; запись преобразованного числа в 'B'

```

Рисунок 2.14: Строки листинга для объяснения

23: 00000101 - адрес в сегменте кода, B8 - опкод инструкции mov для загрузки значения в eax, [0A000000] - адрес B, mov eax, B - приствоеение eax значения B.

24: 00000106 - адрес в сегменте кода, E8 - опкод инструкции call, 91FFFFFF - смещение относительно текущей позиции, по которому находится функция atoi, call atoi - высов функции

25: 0000010B - адрес в сегменте кода, A3 - опкод инструкции mov для записи значения в память, [0A000000] - адрес B, mov [B], eax - запись числа в B

Откроем файл с программой lab7-2.asm и в любой инструкции с двумя операндами удалим один операнд (рис. 2.15).

```

mov ecx ; 'max = C'

```

Рисунок 2.15: Удаляем один из операндов инструкции mov

Выполним трансляцию с получением файла листинга (рис. 2.16).

```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
lab7-2.asm:33: error: invalid combination of opcode and operands

```

Рисунок 2.16: При создании файла листинга выводится сообщение об ошибке

Создается файл листинга lab7-2.lst, в котором добавлено сообщение об ошибке в измененной нами строке (рис. 2.17).

```

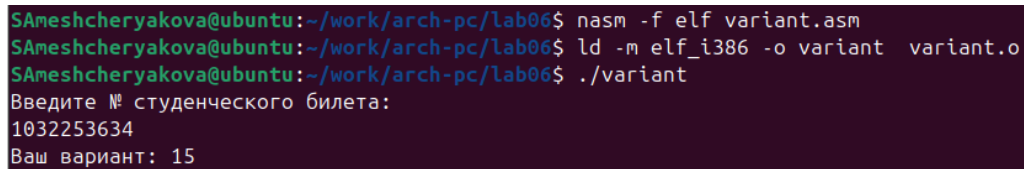
31 00000122 7F06          jg check_B ; если 'A>C', то переход на метку 'check_B',
32 00000124 8B0D[39000000]  mov ecx,[C] ; иначе 'ecx = C'
33                      mov ecx ; 'max = C'
33                      ***** error: invalid combination of opcode and operands

```

Рисунок 2.17: Файл листинга

2.3 Выполнение заданий для самостоятельной работы

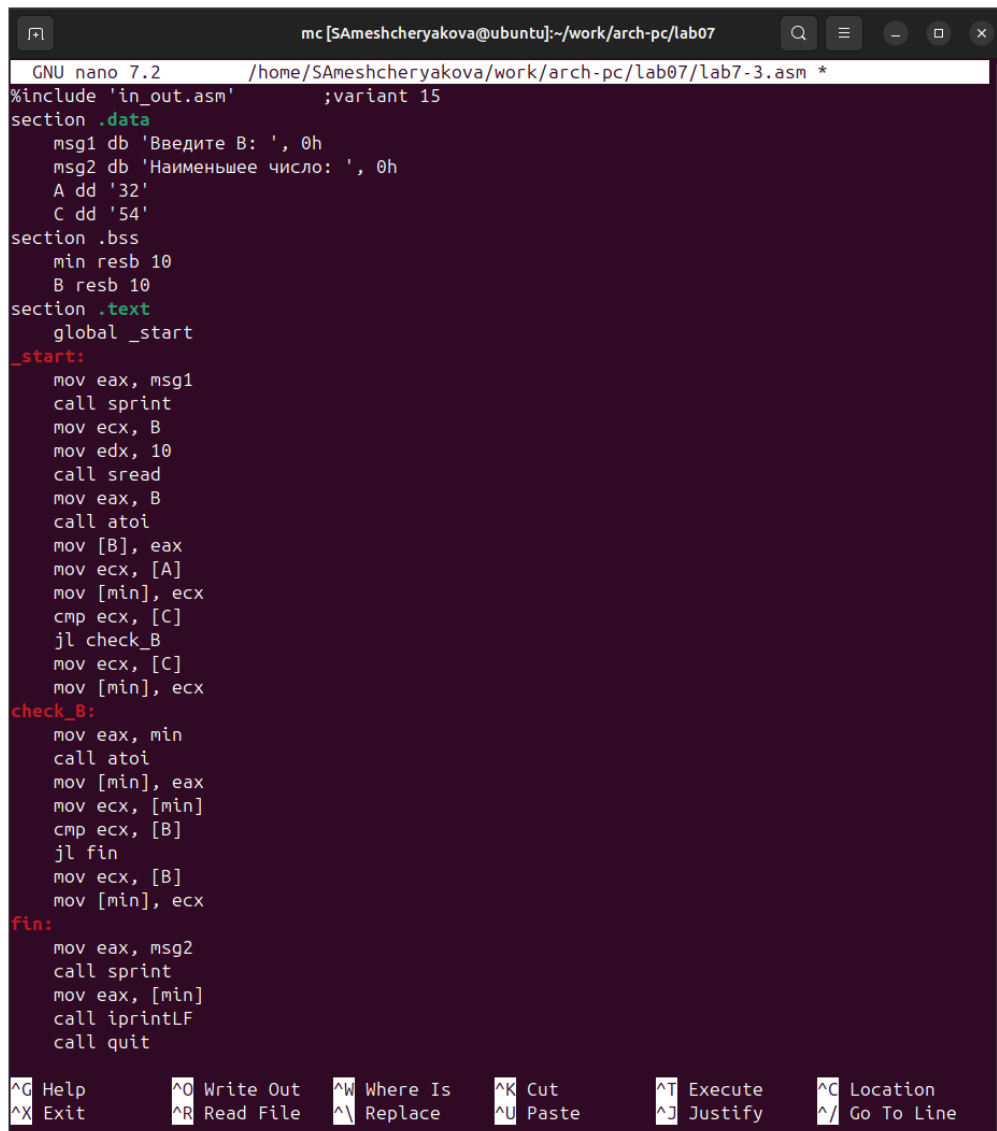
В соответствии с вариантом, полученным при выполнении лабораторной работы №6, выполняем задания 15 варианта (рис. 2.18).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf variant.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant variant.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./variant
Введите № студенческого билета:
1032253634
Ваш вариант: 15
```

Рисунок 2.18: Вариант

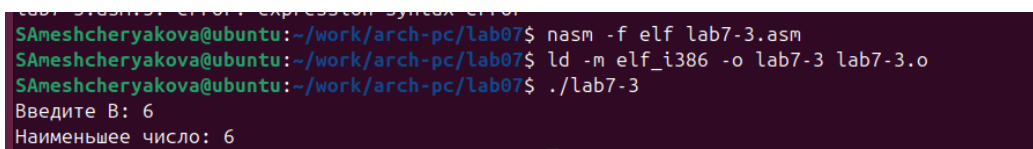
1. Напишем программу нахождения наименьшей из 3 целочисленных переменных a , b и c . Значения $(a, b, c) = (32, 6, 54)$ (рис. 2.19).



```
mc [SAmeshcheryakova@ubuntu]:~/work/arch-pc/lab07
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab07/lab7-3.asm *
#include 'in_out.asm' ;variant 15
section .data
    msg1 db 'Введите B: ', 0h
    msg2 db 'Наименьшее число: ', 0h
    A dd '32'
    C dd '54'
section .bss
    min resb 10
    B resb 10
section .text
    global _start
_start:
    mov eax, msg1
    call sprint
    mov ecx, B
    mov edx, 10
    call sread
    mov eax, B
    call atoi
    mov [B], eax
    mov ecx, [A]
    mov [min], ecx
    cmp ecx, [C]
    jl check_B
    mov ecx, [C]
    mov [min], ecx
check_B:
    mov eax, min
    call atoi
    mov [min], eax
    mov ecx, [min]
    cmp ecx, [B]
    jl fin
    mov ecx, [B]
    mov [min], ecx
fin:
    mov eax, msg2
    call sprint
    mov eax, [min]
    call iprintLF
    call quit
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Рисунок 2.19: Текст программы

Создадим исполняемый файл и проверим его работу (рис. 2.20).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf lab7-3.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-3 lab7-3.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-3
Введите B: 6
Наименьшее число: 6
```

Рисунок 2.20: Запуск программы

Листинг 7.4

```

#include 'in_out.asm'           ;variant 15

section .data
    msg1 db 'XXXXXXXX B: ', 0h
    msg2 db 'XXXXXXXXXXXX XXXXX: ', 0h
    A dd '32'
    C dd '54'

section .bss
    min resb 10
    B resb 10

section .text
    global _start

_start:
    mov eax, msg1
    call sprint
    mov ecx, B
    mov edx, 10
    call sread
    mov eax, B
    call atoi
    mov [B], eax
    mov ecx, [A]
    mov [min], ecx
    cmp ecx, [C]
    jl check_B
    mov ecx, [C]

```

```

    mov [min], ecx

check_B:
    mov eax, min
    call atoi
    mov [min], eax
    mov ecx, [min]
    cmp ecx, [B]
    jl fin
    mov ecx, [B]
    mov [min], ecx

fin:
    mov eax, msg2
    call sprint
    mov eax, [min]
    call iprintLF
    call quit

```

2. Напишем программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений (рис. 2.21).

Вариант 15:

$$f(x) = \begin{cases} a + 10, & x < a \\ x + 10, & x \geq a \end{cases}$$

Значения для подстановки (x,a): (2, 3), (4, 2).

Листинг 7.5

```

#include 'in_out.asm'

SECTION .data
    msg1: DB 'XXXXXXXX a: ', 0h
    msg2: DB 'XXXXXXXX x: ', 0h
    msg3: DB 'F(x) = ', 0h

SECTION .bss
    x:    RESB 10
    a:    RESB 10
    res:  RESB 10

SECTION .text
global _start
_start:
    mov eax,msg1
    call sprint
    mov ecx,a
    mov edx,10
    call sread
    mov eax,a
    call atoi
    mov [a],eax

    mov eax,msg2
    call sprint
    mov ecx,x
    mov edx,10
    call sread

```

```

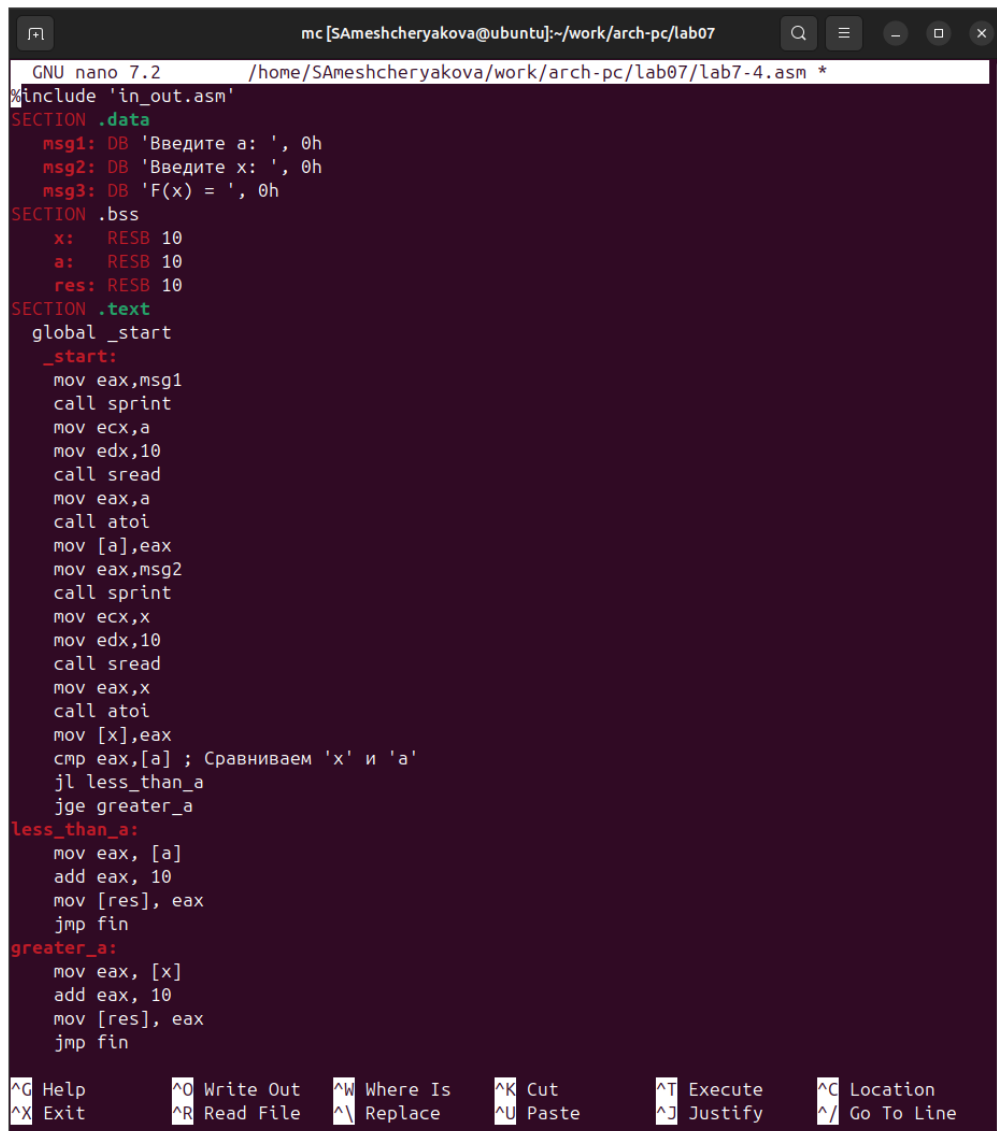
    mov eax, x
    call atoi
    mov [x], eax
    cmp eax, [a]
    jl less_than_a
    jge greater_a

less_than_a:
    mov eax, [a]
    add eax, 10
    mov [res], eax
    jmp fin

greater_a:
    mov eax, [x]
    add eax, 10
    mov [res], eax
    jmp fin

fin:
    mov eax, msg3
    call sprint
    mov eax, [res]
    call iprintLF
    call quit

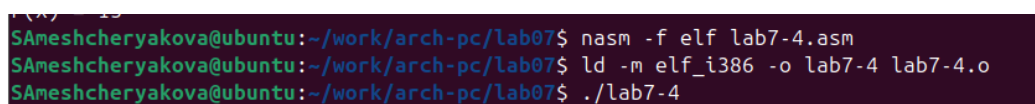
```



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab07/lab7-4.asm *
#include 'in_out.asm'
SECTION .data
msg1: DB 'Введите a: ', 0h
msg2: DB 'Введите x: ', 0h
msg3: DB 'F(x) = ', 0h
SECTION .bss
x: RESB 10
a: RESB 10
res: RESB 10
SECTION .text
global _start
_start:
mov eax,msg1
call sprint
mov ecx,a
mov edx,10
call sread
mov eax,a
call atoi
mov [a],eax
mov eax,msg2
call sprint
mov ecx,x
mov edx,10
call sread
mov eax,x
call atoi
mov [x],eax
cmp eax,[a] ; Сравниваем 'x' и 'a'
jl less_than_a
jge greater_a
less_than_a:
mov eax, [a]
add eax, 10
mov [res], eax
jmp fin
greater_a:
mov eax, [x]
add eax, 10
mov [res], eax
jmp fin
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Рисунок 2.21: Текст программы

Создадим исполняемый файл и проверим его работу (рис. 2.22).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ nasm -f elf lab7-4.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-4 lab7-4.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-4
```

Рисунок 2.22: Запуск программы


```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-4
Введите a: 3
Введите x: 2
F(x) = 13
```

Рисунок 2.23: Запуск программы при (x,a): (2, 3)

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab07$ ./lab7-4
Введите a: 2
Введите x: 4
F(x) = 14
```

Рисунок 2.24: Запуск программы при (x,a): (4, 2)

3 Выводы

Мы познакомились с структурой файла листинга, изучили команды условного и безусловного перехода.